

C++ Map Practice Problems & Solutions

A collection of foundational C++ `std::map` and `std::unordered_map` practice problems with clean implementations.

1. Character / Element Frequency Counter

Goal: Count the frequency of each element in a collection and output the counts sorted by key.

Key Concepts: `std::map` key-value pair accumulation, ordered iteration.

```
#include <iostream>
#include <vector>
#include <map>
using namespace std;

int main() {
    vector<int> vNums = {8, 4, 1, 8, 2, 2, 3, 8, 1, 4, 4, 5 };
    map<int, int> mCounts;

    for (int num : vNums) {
        mCounts[num]++;
    }

    cout << "Element counts (sorted by key):\n";
    for (auto const& mRow : mCounts) {
        cout << mRow.first << ": " << mRow.second << "\n";
    }

    cout << "All done! " << endl;
}
```

2. Find Most Frequent Element (Majority / Mode)

Goal: Find the element that appears most frequently in a vector.

Key Concepts: `std::unordered_map` for $O(1)$ frequency counting, running maximum tracking.

```
#include <iostream>
#include <vector>
#include <unordered_map>
using namespace std;

int findMostFrequent(const vector<int>& vNums) {
    unordered_map<int, int> mFreq;
    int maxCount = 0;
    int mostFrequent = vNums[0];

    for (int num : vNums) {
        mFreq[num]++;
        if (mFreq[num] > maxCount) {
            maxCount = mFreq[num];
            mostFrequent = num;
        }
    }

    return mostFrequent;
}

int main() {
    vector<int> vNums = { 1, 3, 2, 3, 4, 3, 2, 1 };

    cout << "Most frequent element: " << findMostFrequent(vNums) << "\n";

    cout << "All done!" << endl;
}
```

3. Valid Anagram Check

Goal: Given two strings, determine if one is an anagram of the other.

Key Concepts: `std::unordered_map` frequency matching, character counts.

```
#include <iostream>
#include <string>
#include <unordered_map>
using namespace std;

bool isAnagram(string s, string t) {
    if (s.length() != t.length()) return false;

    unordered_map<char, int> mCharCounts;

    for (char c : s) {
        mCharCounts[c]++;
    }

    for (char c : t) {
        mCharCounts[c]--;
        if (mCharCounts[c] < 0) {
            return false;
        }
    }

    return true;
}

int main() {
    string s1 = "listen", t1 = "silent"; //(earth, heart) (dusty, study)
    string s2 = "rat", t2 = "car";

    cout << s1 << " / " << t1 << " -> "
         << (isAnagram(s1, t1) ? "Anagram" : "Not Anagram") << "\n";

    cout << s2 << " / " << t2 << " -> "
         << (isAnagram(s2, t2) ? "Anagram" : "Not Anagram") << "\n";

    cout << "All done!" << endl;
}
```

4. Two Sum (Return Indices)

Goal: Return the 0-based indices of two numbers in a vector that add up to a target value.

Key Concepts: `std::unordered_map` value-to-index mapping for $O(n)$ lookup.

```
#include <iostream>
#include <vector>
#include <unordered_map>
using namespace std;

//Utility: This template function writes a generic
//      map<,> or unordered_map<,>
template <class TMap>
void showMap(const TMap& m, string caption = "") {
    cout << caption << "(Size: " << m.size() << ")" << endl;
    for (const auto& row : m) {
        cout << row.first << " : " << row.second << endl;
    }
}

vector<int> findSolutionPair(const vector<int>& vNums, int target) {
    // This map records where each element lives in the vector:
    // key -> the element's value, value -> its index position
    unordered_map<int, int> mPosition;

    for (int i = 0; i < vNums.size(); ++i) {
        int complement = target - vNums[i];
        //is the complement of the current value in the map?
        if (mPosition.count(complement)) {
            return { mPosition[complement], i };
        }
        mPosition[vNums[i]] = i;
        showMap(mPosition);
    }

    return {};
}

int main() {
    vector<int> vNums = { 2, 8, 7, 1, 15 };
    int target = 9;

    vector<int> indices = findSolutionPair(vNums, target);

    if (!indices.empty()) {
        cout << "Target pair found at indices: ["
             << indices[0] << ", "
             << indices[1] << "]\n";
    }

    cout << "All done!" << endl;
}
```

5. Group Anagrams

Goal: Given an array of strings, group all anagrams together.

Key Concepts: `std::unordered_map` with sorted-string keys and vector values.

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <unordered_map>
using namespace std;

vector<vector<string>> groupAnagrams(const vector<string>& vWords) {
    unordered_map<string, vector<string>> mGroups;

    // Each word is stored under a key formed by sorting its letters.
    // For example, "ate" and "tea" both sort to "aet", so they share
    // the same key and are grouped together in the same value vector.
    // Key   : "aet"
    // Value: {"ate", "tea", "eat"}  -- all anagrams of each other

    for (string word : vWords) {
        string key = word;           //assume word "ate"
        sort(key.begin(), key.end()); //key becomes "aet"
        mGroups[key].push_back(word); //add "ate" to value
    }

    vector<vector<string>> vresult;
    for (auto row : mGroups) {
        vresult.push_back(row.second);
    }
    return vresult;
}

int main() {
    vector<string> words = { "eat", "tea", "tan", "ate", "nat", "bat" };

    vector<vector<string>> vGroups = groupAnagrams(words);

    cout << "Grouped Anagrams:\n";
    for (const auto& group : vGroups) {
        cout << { ";
        for (const string& word : group) {
            cout << word << ", ";
        }
        cout << "} \n";
    }

    cout << "All done!" << endl;
}
```

Sample Output

Grouped Anagrams:

```
{ eat, tea, ate, }
```

```
{ tan, nat, }
```

```
{ bat, }
```